

Eng 4.3. SSDLC

Task 1 - What is SSDLC?

Secure SDLC models aim to introduce security at every stage of the SDLC.

Apart from faster development and reduction of costs, integrating security across the SDLC helps discover and reduce vulnerabilities early, reducing business risk massively.

- Security is a constant concern, improving software quality and security constantly.
- Boosting security education and awareness: all stakeholders know each phase's security recommendations and requirements.
- Flaws are detected early before deployment, reducing the risk of getting hacked or disrupted.
- Costs are reduced, and speed increases, thanks to the early detection and resolution of vulnerabilities. Business risk, brand reputation damage, and fines that could lead to economic disaster for a company are prevented.

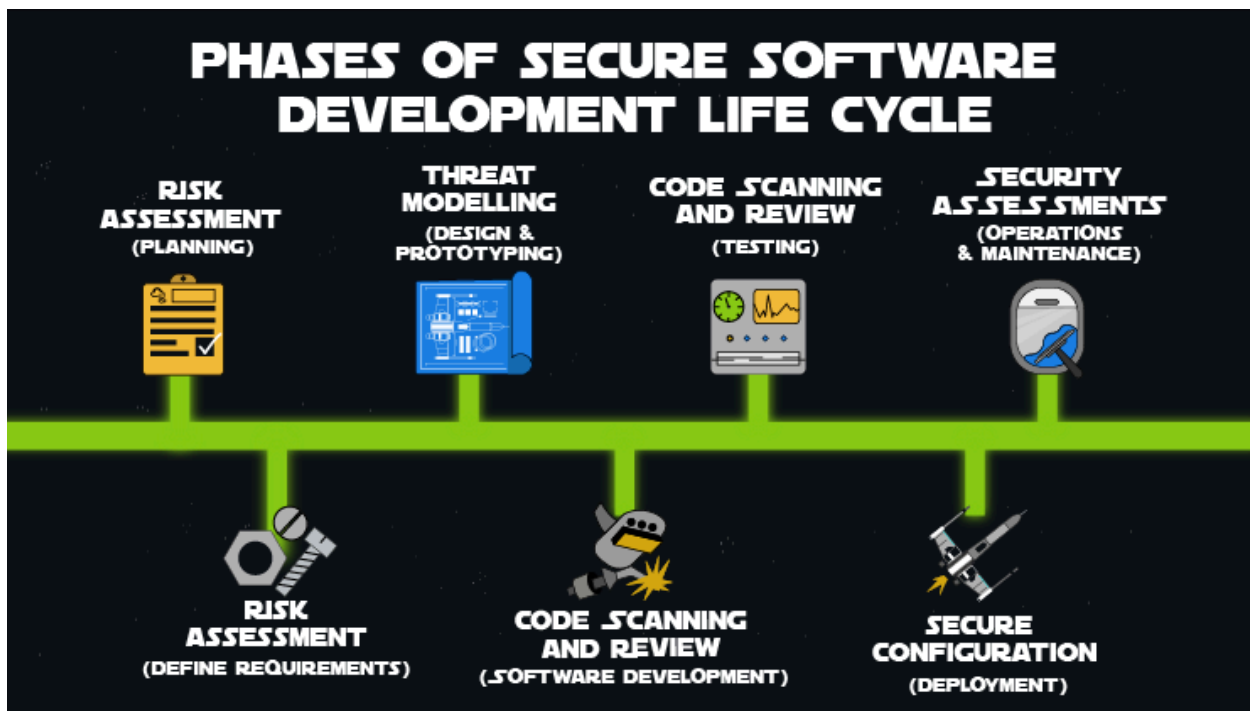
Task 2 - Implementing SSDLC

To help grasp what your security posture is, you can start by doing the following:

- Perform a **gap analysis** to determine what activities and policies exist in your organisation and how effective they are. For example, ensuring policies are in place (what the team does) with security procedures (how the team executes those policies).
- Create **Software Security Initiatives** (SSI) by establishing realistic and achievable goals with defined metrics for success. For example, this could be a Secure Coding Standard, playbooks for handling data, etcetera are tracked using project management tools.

- **Formalise processes** for security activities within your SSI. After starting a program or standard, it is essential to spend an operational period helping engineers get familiarised with it and gather feedback before enforcing it. When performing a gap analysis, every policy should have defined procedures to make them effective.
- **Invest in security training** for engineers as well as appropriate tools. Ensure people are aware of new processes and the tools that will come with them to operationalise them, and invest in training early, ideally before establishing / onboarding the tool.

SSDLC Processes



- **Risk Assessment** during the early stages of SDLC, it is essential to identify security considerations that promote a security by design approach when functional requirements are gathered in the planning and requirements stages. For example, if a user requests a blog entry from a site, the user should not be able to edit the blog or remove unnecessary input fields.
- **Threat Modelling** - is the process of identifying potential threats when there is a lack of appropriate safeguards. It is very effective when following a risk assessment and during the design stage of the SDLC, as Threat Modelling

focuses on what should not happen. In contrast, design requirements state how the software will behave and interact. For example, ensure there is verification when a user requests account information.

- **Code Scanning / Review** - Code reviews can be either manual or automated. Code Scanning or automated code reviews can leverage Static and Dynamic Security testing technologies. These are crucial in the Development stages as code is being written.
- **Security Assessments - Like Penetration Testing & Vulnerability Assessments** are a form of automated testing that can identify critical paths of an application that may lead to exploitation of a vulnerability. However, these are hypothetical as the assessment doesn't carry simulations of those attacks. Pentesting, on the other hand, identifies these flaws and attempts to exploit them to demonstrate validity. Pentests and Vulnerability Assessments are carried out during the Operations & Maintenance phase of the SDLC after a prototype of the application.

There are methodologies to apply the processes in task 5 that will help you navigate and guide you when introducing risk assessment, threat modelling, scanning and testing, and operational assurance. The following tasks will cover these processes in more detail.

Answer the questions below

| What should you understand before implementing Secure SDLC processes?

Security Posture

| During which stages should you perform a Risk Assessment?

Planning and Requirements

| What should be carried out during the design phase?

Threat Modelling

Task 3 - Risk Assessment

Risk refers to the likelihood of a threat being exploited, negatively impacting a resource or the target it affects.

Risk assessment is used to determine the level of the potential threat. Risk identified in the risk assessment process can be reduced or eliminated by applying appropriate controls during the risk mitigation process. Usually, a risk assessment is followed by threat modelling, which will be explained further in the next section.

Types of Risk Assessments

There are several types of Risk assessments best suited for different scenarios. Qualitative Risk Assessment and Quantitative Risk Assessment

5X5 RISK MATRIX

SEVERITY →

	1 Insignificant	2 Minor	3 Moderate	4 Major	5 Death
1 Rare	1	2	3	4	5
2 Unlikely	2	4	6	8	10
3 Possible	3	6	9	12	15
4 Likely	4	8	12	16	20
5 Certain	5	10	15	20	25

↑
LIKELIHOOD

Answer the questions below

| What is a formula to assign a Qualitative Risk level?

Severity x Likelihood

Which type of Risk Assessment assigns numerical values to determine risk?

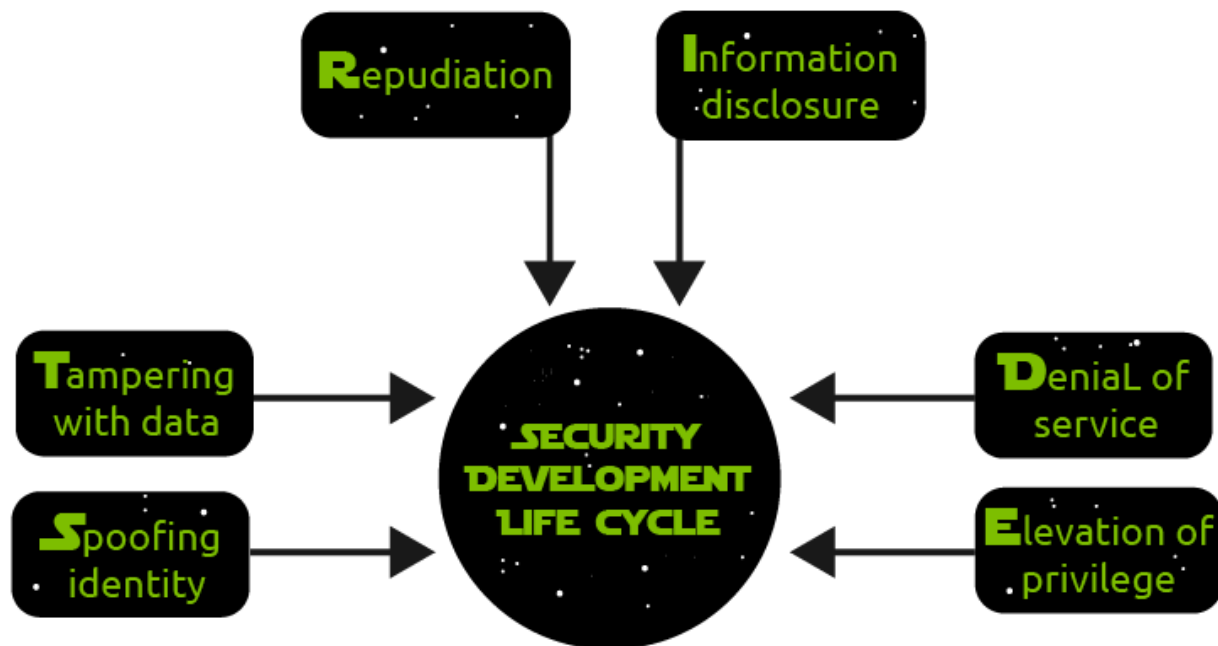
Quantitative Risk Assessment

Task 4 - Threat Modelling

Threat modelling is best integrated into the design phase of an SDLC before any code is written. Threat modelling is a structured process of identifying potential security threats and prioritising techniques to mitigate attacks so that data or assets that have been classified as valuable or of higher risk during risk assessment, such as confidential data, are protected. **STRIDE, DREAD, and PASTA are among the common threat modelling methodologies.**

STRIDE stands for Spoofing, Tampering, Repudiation, Information Disclosure, Denial Of Service, and Elevation/Escalation of Privilege.

- **Spoofing:** Impersonation of a user by a malicious actor, violating the authentication principle. Examples include ARP, IP, and DNS spoofing.
- **Tampering:** Modification of information by an unauthorised user, violating the integrity principle.
- **Repudiation:** Lack of accountability for actions where responsibility cannot be attributed, violating non-repudiability.
- **Information Disclosure:** Violation of confidentiality, such as in data breaches.
- **Denial of Service:** Exhaustion of resources, preventing authorised users from accessing a system, violating availability.
- **Elevation of Privilege:** Gaining unauthorised access by escalating privileges, violating authorisation principles.



DREAD

The abbreviation DREAD stands for five questions about each potential threat: Damage Potential, Reproducibility, Exploitability, Affected Users, and Discoverability. DREAD ranks threats by assigning scores based on their severity and risk probability. Here are its components:

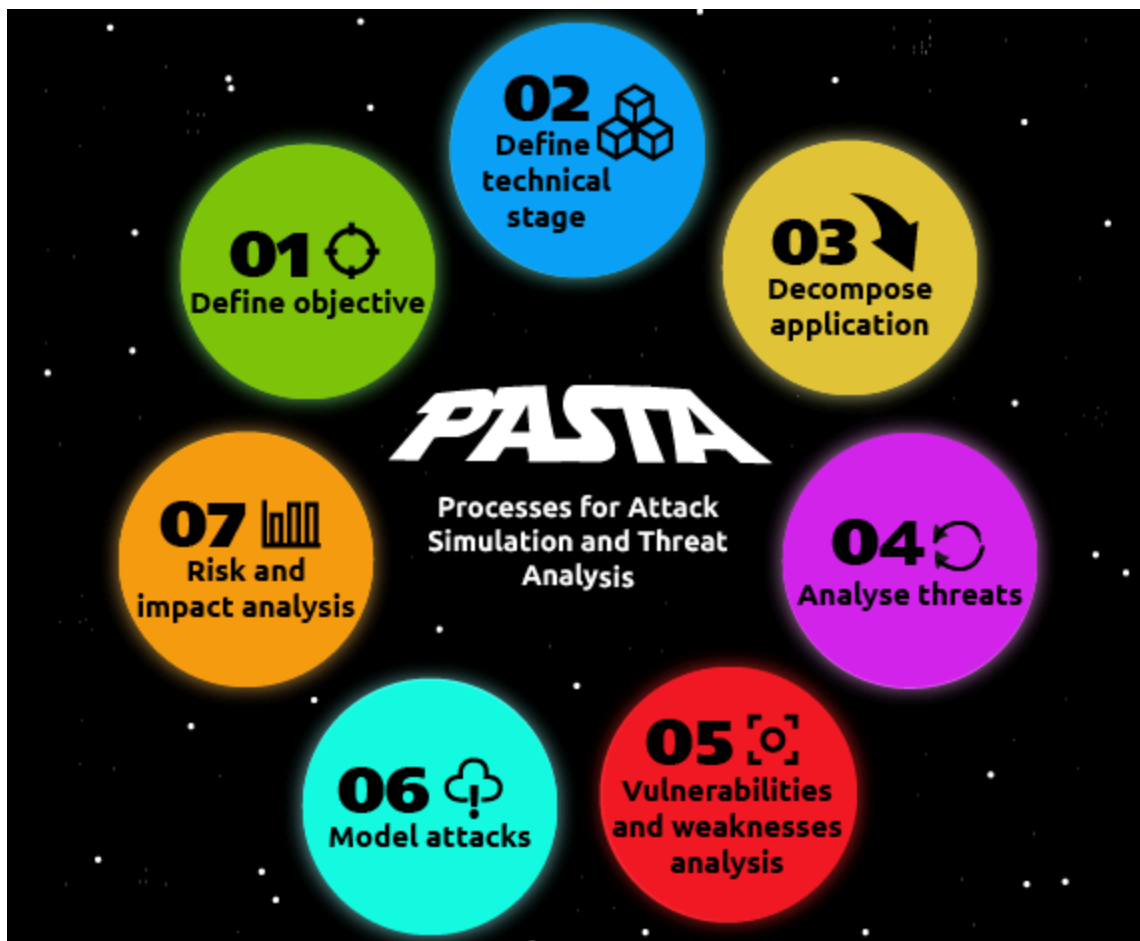
- **Damage:** The potential impact of a threat, scored on a scale of 0–10.
- **Reproducibility:** The complexity of reproducing the threat, scored 0–10.
- **Exploitability:** How easily the threat can be exploited, scored 0–10.
- **Affected Users:** The number of users impacted, scored 0–10.
- **Discoverability:** The ease of discovering the threat, scored 0–10.

PASTA

PASTA stands for Process for Attack Simulation and Threat Analysis. It aligns technical requirements with business objectives and provides a strategic view of threats. PASTA's seven stages are:

- **Define Objectives:** Establish the scope and objectives.

- **Define Technical Scope:** Create architectural diagrams to map the attack surface.
- **Decomposition & Analysis:** Map trust boundaries and evaluate vulnerabilities.
- **Threat Analysis:** Identify potential threats based on intelligence.
- **Vulnerabilities & Weaknesses Analysis:** Assess and mitigate weaknesses.
- **Attack/Exploit Enumeration & Modelling:** Simulate attack paths and vulnerabilities.
- **Risk Impact Analysis:** Document risks and propose mitigation steps.



Answer the questions below

What threat modelling methodology assigns a rating system based on risk probability?

DREAD

| What threat modelling methodology is built upon the CIA triad?

STRIDE

| What threat modelling methodology helps align technical requirements with business objectives?

PASTA

Task 5 - Secure Coding

Code review can be done manually or automated. A manual code review is where an expert analyses and checks the source code by going line by line to identify vulnerabilities.

Code Analysis: Automated Static Application Security Testing (SAST) automatically analyses and checks the source code. **SAST** means Static Application Security Testing, a white box testing method that directly analyses the source code. SAST is used to scan source code for security vulnerabilities. Another type of testing goes hand in hand with SAST, Software Composition Analysis (SCA).

SCA: SCA is used to scan dependencies for security vulnerabilities, helping development teams track and analyse any open-source component brought into a project. SCA is now an essential pillar in security testing as modern applications are increasingly composed of open-source code.

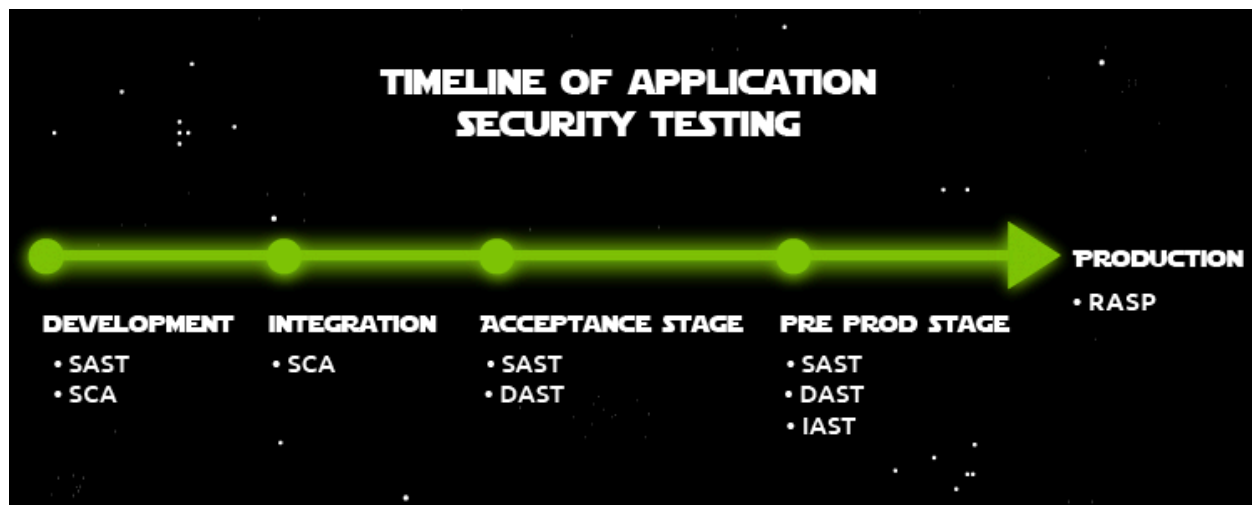
DAST means Dynamic Application Security Testing, a black-box testing method that finds vulnerabilities at runtime. DAST is a tool to scan any web application to find security vulnerabilities. This tool is used to detect vulnerabilities inside a web application that has been deployed to production. DAST tools will always send alerts to the security team assigned for immediate remediation. DAST works by simulating automated attacks on an application, mimicking a malicious attacker. The goal is to find unexpected outcomes or results that attackers could use to compromise an application. Since DAST tools don't have internal information about the application or the source code, they attack just as an external hacker would—with the same limited knowledge and information about the application.

IAST means Interactive Application Security Testing that analyses code for security vulnerabilities while the app is running. It is usually deployed side by side with the main application on the application server. IAST is an application security tool designed for web and mobile applications to detect and report issues even while running.

IAST testing occurs in real-time, just like DAST, while the application runs in the staging environment. IAST can identify the line of code causing security issues and quickly inform the developer for immediate remediation. IAST checks the source code similar to SAST, but at the post-build stage, unlike SAST, which occurs during development. IAST agents are typically deployed on the application servers. When the DAST scanner performs its work by reporting a vulnerability, the deployed IAST agent will now return a line number of the issue from the source code.dangerous.

RASP stands for Runtime Application Self Protection. RASP is a runtime application integrated into an application to analyse inward and outward traffic and end-user behavioural patterns to prevent security attacks. This tool is different from the other tools as RASP is used after product release, making it a more security-focused tool when compared to the others that are known for testing. RASP is deployed to a web or application server next to the main application while running to monitor and analyse the inward and outward traffic behaviour. Immediately once an issue is found, RASP will send alerts to the security team and immediately block access to the individual making a request. When you deploy RASP, it will secure the whole application against different attacks.

Choosing tools: SAST, DAST, and IAST are great tools that complement each other. A key strength of DAST is that it identifies runtime issues—weaknesses that aren't discoverable when an application isn't running. SAST is excellent at identifying vulnerabilities while code is being written. Additionally, DAST looks at how an application responds to an attack, providing helpful insight into how likely it would be for that vulnerability to be manipulated. AST enables DevSecOps and supports continuous testing, monitoring, assessment, and validation in real-time. IAST helps prioritise and alert on vital critical risks, as defined by business goals and application security needs.



Above is a standard timeline when choosing when to implement security testing tooling. As described earlier, SAST is a static approach; there is no need for a running application. Therefore, SAST can be implemented in the earliest development stages. SCA typically work best for Identifying open source components, like packages via dependencies. It is also beneficial for identifying open-source licenses being used, especially from a legal risk perspective. Because of this, you can implement governance and control by enforcing security and license policies across the different stages of SDLC.

On the other hand, DAST relies on the execution of the application, so integrating it in the pipeline is not as straightforward as SAST. DAST is typically implemented after the acceptance stage of the code and pre-production stage. This is where the application starts running in testing environments (e.g. sandbox/staging) similar to IAST. DAST and IAST reside on the far right of your pipeline because the execution of your application is required for DAST tools to do their work. This can become quite time-consuming, significantly if your application has grown over time. RASP helps uncover third-party packages and associated vulnerabilities at runtime, more effectively enabling developers to prioritise the remediation and mitigation of their highest pressing vulnerabilities. This is usually enabled in the production stage as security vulnerabilities are evaluated during the runtime of the application.

Answer the questions below

| Is it recommended to use SAST analysis at the beginning of the SDLC?

YES

| Which type of code analysis uses the black-box method?

DAST

| Which type of code analysis uses the white-box method?

SAST

Task 6 - Security Assessments

A security assessment plays a primary role in achieving security in SDLC and should be implemented in all phases where possible. Security testing assesses a system, software or web application for vulnerabilities and other attack vectors.

Vulnerability Assessment

Pros	Cons
• Suitable for quickly identifying potential vulnerabilities	• Can produce a large number of reports
• Part of the Penetration Test	• Quality depends on the tool used
• Better for Budget, they are cheaper than Pentests	• Real-life scenarios for vulnerabilities are not considered (it could be behind a proxy or only exploitable with social engineering/credentials)
	• The low-risk vulnerability may be used as part of a more powerful attack.

Penetration testing

Pros	Cons
Tester shows organisations what an attacker could do.	Very Expensive
How any vulnerabilities could be used against it by attackers – the real risk	Requires extensive planning and time to carry out the tests
Can be shown to the customer	

Answer the questions below

Which form of assessment is more budget-friendly and takes less time?

Vulnerability Assessment

Which type of assessment identifies vulnerabilities and attempts to exploit them?

Penetration Testing

When do you typically carry out Vulnerability Assessments or Pentests?

Operations & Maintenance

Task 7 - SSDLC Methodologies

SSDLC Methodologies: There are several methodologies for SDLC; some examples of widely used methodologies are:

- Microsoft's Security Development Lifecycle (SDL)
- OWASP Secure Software Development Life Cycle Project (S-SDLC)
- Software Security Touchpoints

SDL is a collection of mandatory security activities grouped by the traditional software development lifecycle phases. Data is collected to assess training effectiveness. In-process metrics are used to confirm process compliance. Post-release metrics are used to guide future changes. SDL places heavy emphasis on understanding the cause and effect of security vulnerabilities. A development team must complete the mandatory security activities to comply with the Microsoft SDL process. You can implement SDL by following these practices:

Practice	Why?
Provide Training	Engineers, program and product managers must understand security basics and know how to build security into software and services to make products more secure while still addressing business needs and delivering user value. Practical training will complement and re-enforce security policies, SDL practices, standards, and software

	security requirements and be guided by insights derived through data or newly available technical capabilities.
Define Security Requirements	Considering security and privacy is a fundamental aspect of developing highly secure applications and systems. Regardless of the development methodology being used, must continually update security requirements to reflect changes in required functionality and changes to the threat landscape.
Define Metrics and Compliance Reporting	It is essential to define the minimum acceptable levels of security quality and hold engineering teams accountable for meeting those criteria. Defining these early helps a team understand risks associated with security issues, identify and fix security defects during development, and apply the standards throughout the entire project.
Perform Threat Modeling	This practice allows development teams to consider, document, and discuss the security implications of designs in their planned operational environment and in a structured fashion. Applying a structured approach to threat scenarios helps a team more effectively and less expensively identify security vulnerabilities, determine risks from those threats, make security feature selections and establish appropriate mitigations.
Establish Design Requirements	The SDL is typically thought of as assurance activities that help engineers implement "secure features", in that the features are well-engineered concerning security. Engineers will normally rely on security features, such as cryptography, authentication, logging, etc. In many cases, the selection or implementation of security features has proven so complicated that design or implementation choices are likely to result in vulnerabilities. Therefore, it's crucially essential to apply these consistently and with a consistent understanding of their protection.
Define and Use Cryptography Standards	It's critically important to ensure all data, including security-sensitive information and management and control data, is protected from unintended disclosure or alteration when transmitted or stored. Encryption is typically used to achieve this. E.g., only use industry-vetted encryption libraries and ensure they're implemented to allow them to be easily replaced if needed.

Manage the Security Risk of Using Third-Party Components	When selecting third-party components to use, it's essential to understand the impact of a security vulnerability on the security of the more extensive system into which they are integrated. Having an accurate inventory of third-party components and a plan to respond when new vulnerabilities are discovered will go a long way toward mitigating this risk.
Use Approved Tools	Define and publish a list of approved tools and their associated security checks, such as compiler/linker options and warnings. Engineers should strive to use the latest version of approved tools, such as compiler versions, and take advantage of new security analysis functionality and protections.
Perform Security Testing (SAST, DAST, IAST)	Analysing the source code before compilation provides a highly scalable method of security code review and helps ensure that secure coding policies are being followed. The same goes for performing run-time verification of your fully compiled or packaged software checks functionality that is only apparent when all components are integrated and running.
Perform Security Assessments: Vulnerability Assessment & Penetration Testing	The objective of a penetration test is to uncover potential vulnerabilities resulting from coding errors, system configuration faults, or other operational deployment weaknesses. As such, the test typically finds the widest variety of vulnerabilities.
Establish a Standard Incident Response Process	Preparing an Incident Response Plan is crucial for helping to address new threats that can emerge over time. The plan should include whom to contact in case of a security emergency and establish the protocol for security servicing, including methods for code inherited from other groups within the organisation and for third-party code.

The Software Assurance Maturity Model (SAMM) is an open framework to help organisations formulate and implement a software security strategy tailored to the organisation's specific risks. It helps to evaluate an organisation's existing software security practices, build a software security assurance program, demonstrate improvements to that program, and define and measure security activities for an organisation. SAMM helps explain objectives, actions, results,

success metrics, costs etc. An example would be a security scorecard for gap analysis, for instance, in a particular area, like endpoint protection. It aims to answer "How well are we doing and where do we want to get to?".

Another critical security model is the Building Security In Maturity Model (BSIMM). BSIMM is a study of real-world software security initiatives and reflects the current state of software security. BSIMM can be described as a "measuring stick" to understand your security posture by providing a comparison of other companies' security states. In other words, it does not tell you what you should do but rather what you are doing wrong.

Answer the questions below

What methodology follows a set of mandatory procedures embedded in the SDLC?

Microsoft SDL

What Maturity Model helps you measure tailored risks facing your organisation?

SAMM

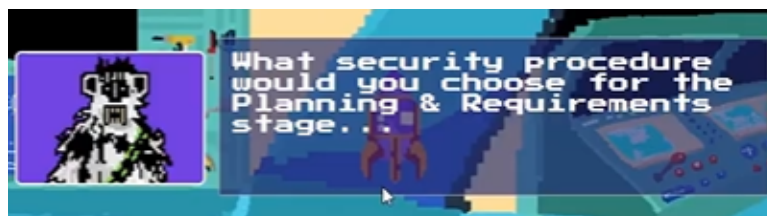
What maturity model acts as a measuring stick to determine your security posture?

BSIMM

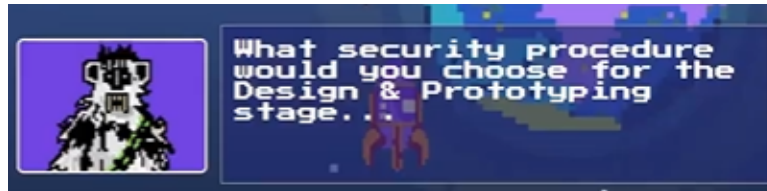
Task 8 - Secure Space Lifecycle

Answer the questions below

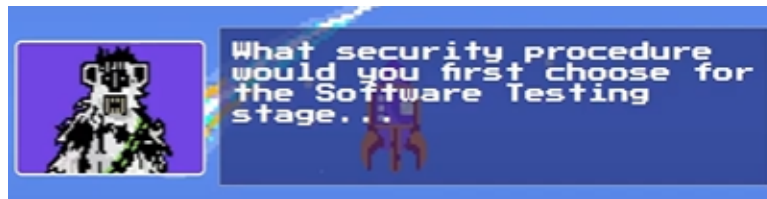
What is the flag? THM{D0-A-Barr3I-R011}



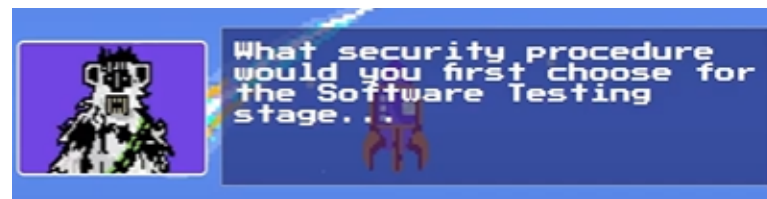
Risk Management



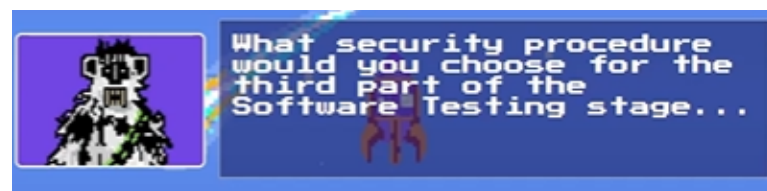
Threat Modelling



SAST



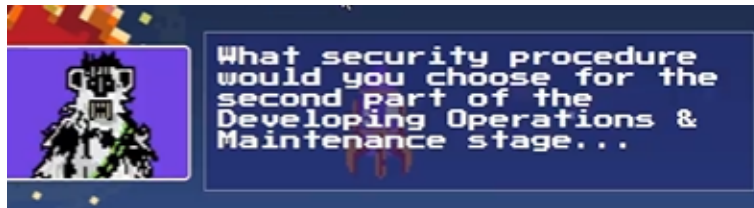
DAST



Code Review



Secure Config



Security Assessment

Task 9 - Conclusion

SSDLC (Secure Software Development Life Cycle) is a software development process that integrates security activities into every phase of the traditional SDLC. Its goal is to identify, prevent, and remediate security vulnerabilities throughout the development lifecycle rather than after deployment. Why SSDLC Matters

- Reduces security risks early in development
- Lowers the cost of fixing vulnerabilities
- Helps meet compliance and regulatory requirements
- Improves software quality and resilience
- Supports DevSecOps practices

SSDLC Phases

1. Requirements Phase: Security requirements are defined alongside business requirements. **Activities:**

- Identify security objectives
- Define compliance requirements (e.g., GDPR, PCI DSS)
- Perform risk assessments
- Establish security policies

2. Design Phase: Security is built into the application's architecture. **Activities:**

- Threat modeling
- Attack surface analysis

- Secure architecture reviews
- Define authentication and authorization mechanisms

Common Framework:

- OWASP Threat Modeling

3. Development Phase: Developers follow secure coding practices. **Activities:**

- Secure coding standards
- Code reviews
- Static Application Security Testing (SAST)
- Secrets management

Tools:

- Semgrep
- CodeQL
- SonarQube
- Checkmarx

4. Testing Phase: Security testing is performed before release. **Activities:**

- Dynamic Application Security Testing (DAST)
- Penetration testing
- Vulnerability scanning
- Interactive Application Security Testing (IAST)

Tools:

- OWASP ZAP
- Burp Suite
- Nessus

5. Deployment Phase: Applications are securely released to production.

Activities:

- Secure configuration management

- Infrastructure hardening
- Container security checks
- CI/CD security controls

6. Maintenance & Operations Phase: Security continues after deployment.

Activities:

- Patch management
- Log monitoring
- Incident response
- Continuous vulnerability assessments

SSDLC Security Activities by Phase

Phase	Security Activities
Requirements	Risk assessment, security requirements
Design	Threat modeling, architecture review
Development	Secure coding, SAST, code review
Testing	DAST, penetration testing, vulnerability scanning
Deployment	Hardening, secure configurations
Maintenance	Monitoring, patching, incident response

Common SSDLC Tools

Category	Examples
SAST	Semgrep, Checkmarx, CodeQL, SonarQube
DAST	OWASP ZAP, Burp Suite
SCA	Snyk, Dependabot, Mend
Container Security	Trivy, Clair
Secrets Detection	GitLeaks, TruffleHog
Vulnerability Scanning	Nessus, OpenVAS

SSDLC vs Traditional SDLC

Traditional SDLC	SSDLC
Security added late	Security integrated throughout
Vulnerabilities found after development	Vulnerabilities found early
Higher remediation costs	Lower remediation costs
Reactive approach	Proactive approach

SSDLC is the practice of embedding security into every stage of software development—from requirements gathering to maintenance—to build secure applications by design.